Comparison between SAT, ILP and CP approaches to N-Queens problem

Vu Hai Linh[a,*]

[a]University of Engineering and Technology
[a]Vietname National University, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT solving
ILP
CP

ABSTRACT

The N-Queens problem is a fundamental benchmark in combinatorial optimization. This paper presents a comparative empirical study of three distinct solving paradigms: Boolean Satisfiability (SAT), Integer Linear Programming (ILP), and Constraint Programming (CP). Each paradigm is evaluated based on model formulation complexity, scalability, memory consumption, and execution time as the board size N scales. We formulate N-Queens as CNF clauses for SAT solvers, 0-1 linear constraints for ILP solvers, and global constraints (such as `alldifferent`) for CP engines. Experimental results demonstrate that while SAT solvers offer rapid propagation for moderate instance sizes, clause space expansion limits scalability for large N. Conversely, CP formulations achieve superior model conciseness and execution efficiency due to domain-specific filtering algorithms, while ILP provides predictable performance bounded by linear relaxation. Based on these evaluations, we provide practical selection criteria for choosing the optimal solving approach under varying computational constraints.

1. Introduction

The n-queens problem is a classic puzzle in the field of artificial intelligence and combinatorial mathematics. It involves placing n queens on a standard square board of side n in such a way that no two queens can attack each other. This means that no two queens can share the same row, column, or diagonal. First proposed in the 19th century, the problem has become a significant topic of study due to its simplicity and broad applications in areas such as scheduling, optimization, and algorithm design.

This paper proposes three distinct strategies to address N-Queens Problem namely SAT solving, ILP and CP as well as evaluating the effectiveness of aforementioned approaches regarding formulation complexity, memory consumption and execution time on different size of board.

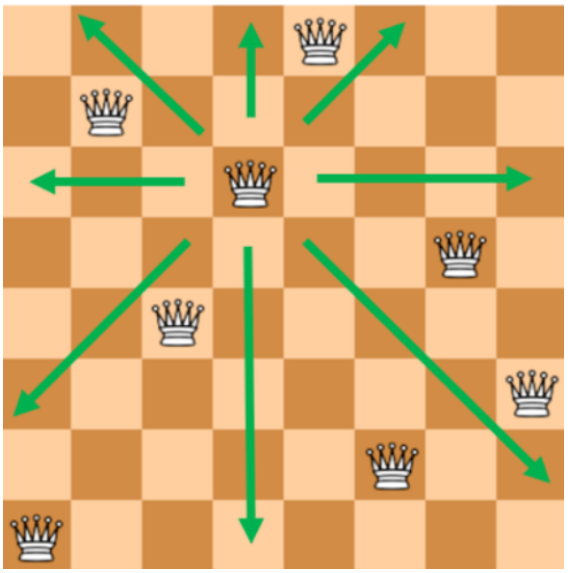
2. Problem Definition

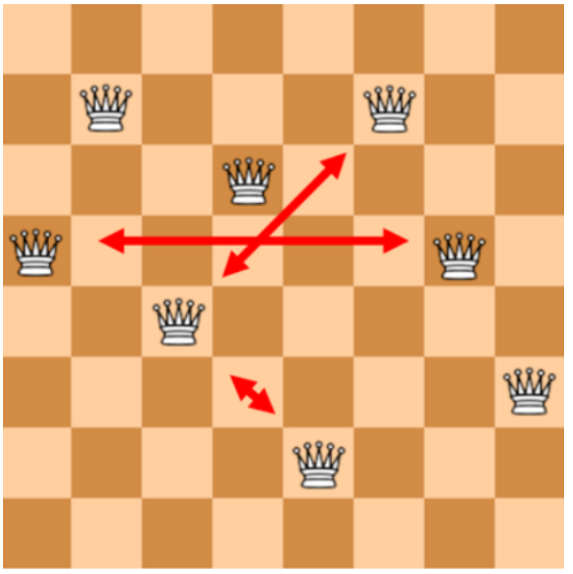
The problem is set on an $n \times n$ chessboard, where n is a positive integer. There are n queens to be placed on the board. The main objective is to place all n queens on the chessboard so that no two queens threaten each other. This means:

- Rule 1: No two queens can be placed in the same row.
- Rule 2: No two queens can be placed in the same column.
- Rule 3: No two queens can be placed on the same diagonal.

The above rules can be interpreted as:

- There is exactly one queen in each row.
- There is exactly one queen in each column.
- There is at most one queen in each diagonal.





The figures above show the difference between a valid solution and an invalid solution for the n-queens problem. In the valid solution, no queens are attacking each other as is explained for the queen in D3. In the invalid solution, there are 4 queens attacking each other (A4, C5, E7, F6, and G4).

3. SAT-based Approach

Boolean Satisfiability, commonly referred to as SAT is a fundamental problem in computational theory and logic. It involves determining whether there exists an assignment of truth values (true/false) to variables in a logical formula that makes the formula true. This formula is typically represented in conjunctive normal form (CNF), where it is composed of a conjunction of clauses, with each clause being a disjunction of literals. A SAT Solver is an algorithmic tool designed to solve SAT problems. It explores possible assignments systematically or heuristically to find a satisfying assignment or prove that none exists. SAT Solvers have broad applications in fields such as artificial intelligence, verification, and optimization, due to their ability to handle complex logical conditions efficiently. There are many SAT solvers available, such as Glucose, MiniSat, Lingeling, Sat4j, and many more. These solvers have different strengths and weaknesses, making them suitable for different types of problems.

The SAT solver takes a clause in CNF as input and outputs whether the formula is satisfiable (SAT) or unsatisfiable (UNSAT). It also provides the potential values that make the formula true.

- INPUT: A propositional logic formula, represented as in the form of conjunctive normal form.
- OUTPUT:
  – SAT: if there exists an assignment of truth values (true/false) to the propositional logic variables that makes the original propositional logic formula true.
  – UNSAT: when all possible assignments of truth values (true/false) to the propositional logic variables make the original propositional logic formula false.

For example:

$$\begin{aligned} F_1 &= (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3) \wedge (\neg x_2 \vee x_1) \\ F_2 &= \neg x_1 \wedge (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \end{aligned} \tag{1}$$

The formula F_1 is true when $x_1 = \text{false}$, $x_2 = \text{false}$, $x_3 = \text{true}$ so F_1 is SAT. However, F_2 is UNSAT because it is false irrespective of any set of values (x_1, x_2).

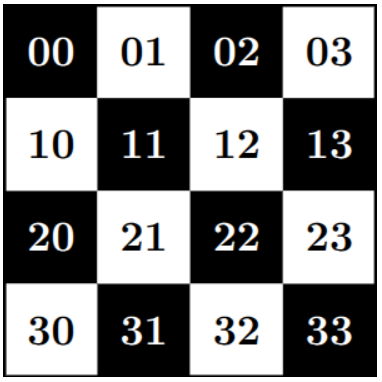
24021547@vnu.edu.vn (V.H. Linh)
ORCID(s): 0000-0000-0000-0000 (V.H. Linh)

3.1. SAT-based encoding

SAT Encoding is an approach of representing problems through propositional logic formulas and applying SAT Solvers to solve these propositional logic formulas.

3.1.1. Pairwise Encoding

Each position (i, j) (the i-th column and the j-th row) in the $n \times n$ board is represented by a variable $B_{i,j}$ for $0 \leq i, j < n$. $B_{i,j}$ is true if a queen is at position (i, j) and it is false otherwise. The 4×4 board below is used as an example. $B_{2,3}$ is true if a queen is at position $(2, 3)$.



The constraints of N-Queens Problem are encoded as:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row.

At most one queen in each row

$$\bigwedge_{i=0}^{n-1} \bigwedge_{j=0}^{n-2} \bigwedge_{k=j+1}^{n-1} (\neg B_{ij} \vee \neg B_{ik}) \tag{2}$$

For example:

$$(\neg B_{00} \vee \neg B_{01}) \wedge (\neg B_{00} \vee \neg B_{02}) \wedge (\neg B_{00} \vee \neg B_{03}) \wedge \ldots \wedge (\neg B_{21} \vee \neg B_{22}) \wedge (\neg B_{20} \vee \neg B_{23}) \wedge \cdots \wedge (\neg B_{32} \vee \neg B_{33})$$

At least one queen in each row

$$\bigwedge_{i=0}^{n-1} (B_{i0} \vee B_{i1} \vee \cdots \vee B_{i,n-1}) \tag{3}$$

For example:

$$(B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge (B_{10} \vee B_{11} \vee B_{12} \vee B_{13}) \wedge (B_{20} \vee B_{21} \vee B_{22} \vee B_{23}) \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})$$

$\Rightarrow$ Exactly one queen in each row:

$$((\neg B_{00} \vee \neg B_{01}) \wedge \cdots \wedge (\neg B_{32} \vee \neg B_{33})) \wedge (B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge \ldots \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})$$

(b) Exactly one queen per column

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each column.

At most one queen in each column

$$\bigwedge_{j=0}^{n-1}\bigwedge_{i=0}^{n-2}\bigwedge_{k=i+1}^{n-1}(\neg B_{ij}\vee\neg B_{kj}) \tag{4}$$

For example:

$$(\neg B_{00}\vee\neg B_{10})\wedge(\neg B_{00}\vee\neg B_{20})\wedge(\neg B_{00}\vee\neg B_{30})\wedge\ldots \\ \wedge(\neg B_{12}\vee\neg B_{22})\wedge(\neg B_{03}\vee\neg B_{33})\wedge\cdots\wedge(\neg B_{23}\vee\neg B_{33})$$

At least one queen in each column

$$\bigwedge_{j=0}^{n-1}(B_{0j}\vee B_{1j}\vee\cdots\vee B_{n-1,j}) \tag{5}$$

For example:

$$(B_{00}\vee B_{10}\vee B_{20}\vee B_{30})\wedge(B_{01}\vee B_{11}\vee B_{21}\vee B_{31})\wedge(B_{02}\vee B_{12}\vee B_{22}\vee B_{32})\wedge(B_{03}\vee B_{13}\vee B_{23}\vee B_{33})$$

$\Rightarrow$ Exactly one queen in each column:

$$((\neg B_{00}\vee\neg B_{10})\wedge\cdots\wedge(\neg B_{23}\vee\neg B_{33}))\wedge(B_{00}\vee B_{10}\vee B_{20}\vee B_{30})\wedge\ldots \\ \wedge(B_{03}\vee B_{13}\vee B_{23}\vee B_{33})$$

(c) At most one queen in each diagonal

For this constraint, we consider the main diagonals and anti-diagonals (and their parallel lines) respectively.

At most one queen in the main diagonal and their parallel lines

$$\bigwedge_{k=1-n}^{n-1}\bigwedge_{\substack{0\le i<i'<n\\0\le j<j'<n\\ i-j=i'-j'=k}}(\neg B_{ij}\vee\neg B_{i'j'}) \tag{6}$$

For example:

$$(\neg B_{02}\vee\neg B_{13})\wedge(\neg B_{01}\vee\neg B_{12})\wedge\cdots\wedge(\neg B_{11}\vee\neg B_{22})\wedge\cdots\wedge(\neg B_{20}\vee\neg B_{31})$$

At most one queen in the anti-diagonal and their parallel lines

$$\bigwedge_{k=1}^{2n-3}\bigwedge_{\substack{0\le i<i'<n\\0\le j<j'<n\\ i+j=i'+j'=k}}(\neg B_{ij}\vee\neg B_{i'j'}) \tag{7}$$

For example:

$$(\neg B_{01}\vee\neg B_{10})\wedge(\neg B_{02}\vee\neg B_{11})\wedge\cdots\wedge(\neg B_{03}\vee\neg B_{21})\wedge\cdots\wedge(\neg B_{23}\vee\neg B_{32})$$

This method requires $n\times n$ variables as well as creating $\binom{m}{2}+1$ clauses where m is the number of variables in each constraint. For example, row 0 requires 4 variables representing 4 positions and creates 7 clauses. Despite its simplicity, the pairwise encoding creates an excessive amount of clauses as the side length of the board increments, which reduces the effectiveness of the solver. In the next section, we will discuss other approaches that are more efficient with large boards.

3.1.2. Binary Encoding

In binary encoding (logarithmic encoding), auxiliary boolean variables are introduced to encode row, column, and diagonal position indices as binary strings. Each position (i, j) on the $n \times n$ board is represented by a decision variable $B_{i,j}$ for $0 \leq i, j < n$, where $B_{i,j}$ is true if a queen is at position (i, j) and false otherwise. Additionally, $k = \lceil \log_2 n \rceil$ auxiliary variables $Y_0, Y_1, \ldots, Y_{k-1}$ are defined to map position indices into binary patterns.

The constraints of the N-Queens Problem are encoded as:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row and combine them by AND.

At most one queen in each row

$$\bigwedge_{i=0}^{n-1} \bigwedge_{j=0}^{n-1} \bigwedge_{p=i\cdot k}^{i\cdot k+k-1} (\neg B_{ij} \vee \varphi(j, p)) \tag{8}$$

For example in row_0:

$$(B_{00} \rightarrow \neg Y_0 \wedge \neg Y_1) \wedge (B_{01} \rightarrow Y_0 \wedge \neg Y_1) \wedge (B_{02} \rightarrow \neg Y_0 \wedge Y_1) \wedge (B_{03} \rightarrow Y_0 \wedge Y_1)$$

At least one queen in each row

$$\bigwedge_{i=0}^{n-1} (B_{i0} \vee B_{i1} \vee \cdots \vee B_{i,n-1}) \tag{9}$$

For example:

$$(B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge (B_{10} \vee B_{11} \vee B_{12} \vee B_{13}) \wedge (B_{20} \vee B_{21} \vee B_{22} \vee B_{23}) \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})$$

$\Rightarrow$ Exactly one queen in each row:

$$\begin{aligned}(B_{00} \rightarrow \neg Y_0 \wedge \neg Y_1) \wedge \ldots \wedge (B_{03} \rightarrow Y_0 \wedge Y_1) \\ \wedge (B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge \cdots \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})\end{aligned}$$

(b) Exactly one queen per column

As same as above, this constraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column and combine them by AND.

At most one queen in each column

$$\bigwedge_{j=0}^{n-1} \bigwedge_{i=0}^{n-1} \bigwedge_{p=j\cdot k}^{j\cdot k+k-1} (\neg B_{ij} \vee \varphi(i, p)) \tag{10}$$

For example in column_0:

$$(B_{00} \rightarrow \neg Y_0 \wedge \neg Y_1) \wedge (B_{10} \rightarrow Y_0 \wedge \neg Y_1) \wedge (B_{20} \rightarrow \neg Y_0 \wedge Y_1) \wedge (B_{30} \rightarrow Y_0 \wedge Y_1)$$

At least one queen in each column

$$\bigwedge_{j=0}^{n-1} (B_{0j} \vee B_{1j} \vee \cdots \vee B_{n-1,j}) \tag{11}$$

For example:

$$(B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge (B_{01} \vee B_{11} \vee B_{21} \vee B_{31}) \wedge (B_{02} \vee B_{12} \vee B_{22} \vee B_{32}) \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})$$

$\Rightarrow$ Exactly one queen in each column:

$$(B_{00} \to \neg Y_0 \wedge \neg Y_1) \wedge \ldots \wedge (B_{30} \to Y_0 \wedge Y_1) \\ \wedge (B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge \cdots \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})$$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parallel to them respectively.

At most one queen in the main diagonal and their parallel lines

$$\bigwedge_{q=1-n}^{n-1} \bigwedge_{\substack{0\le i<i'<n \\ 0\le j<j'<n \\ i-j=i'-j'=q}} \bigwedge_{p=j\times k}^{j\times k+k-1} (\neg B_{ij} \vee \varphi(i,p)) \tag{12}$$

For example the diagonal $B_{00}, B_{11}, B_{22}, B_{33}$:

$$(B_{00} \to \neg Y_0 \wedge \neg Y_1) \wedge (B_{11} \to Y_0 \wedge \neg Y_1) \wedge (B_{22} \to \neg Y_0 \wedge Y_1) \wedge (B_{33} \to Y_0 \wedge Y_1)$$

At most one queen in the anti-diagonal and their parallel lines

$$\bigwedge_{k=1}^{2n-3} \bigwedge_{\substack{0\le i<i'<n \\ 0\le j<j'<n \\ i+j=i'+j'=k}} \bigwedge_{p=j\times k}^{j\times k+k-1} (\neg B_{ij} \vee \varphi(i,p)) \tag{13}$$

For example B_{02}, B_{20}, B_{11}:

$$(B_{02} \to \neg Y_0 \wedge \neg Y_1) \wedge (B_{20} \to Y_0 \wedge \neg Y_1) \wedge (B_{11} \to \neg Y_0 \wedge Y_1)$$

By using $k = \lceil \log_2 n \rceil$ auxiliary bit variables, binary encoding generates $O(n \log_2 n)$ clauses per constraint line instead of $O(n^2)$. This logarithmic reduction in clause generation significantly improves solver performance on larger board sizes n.

3.1.3. Commander Encoding

In commander encoding, decision variables $B_{i,j}$ represent whether a queen is placed at position (i, j) on the $n \times n$ board for $0 \le i, j < n$. To reduce clause growth, positions in each constraint line are partitioned into $g = \lceil \sqrt{n} \rceil$ disjoint groups $G_1, G_2, \ldots, G_g$, and an auxiliary commander variable c_k is introduced for each group G_k. A commander variable c_k is true if at least one decision variable in G_k is true, and false if all decision variables in G_k are false.

The constraints of the N-Queens Problem are encoded as:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row and combine them by AND. The "at most one" constraint is encoded as at most one commander is true; if a commander is true, there exists exactly one queen located in a position partition to that commander; if a commander is false, no queen located in that partition. Nevertheless, "at least one" constraint remains the same as pairwise encoding.

At most one queen in each row

$$\bigwedge_{i=0}^{n-1}\Bigg(\bigwedge_{1\le k<k'\le g}(\neg c_{i,k}\vee\neg c_{i,k'})\wedge\bigwedge_{k=1}^{g}\Big((c_{i,k}\rightarrow\bigvee_{j\in G_k}B_{ij})\wedge \bigwedge_{\substack{j,j'\in G_k\\ j<j'}}(c_{i,k}\rightarrow(\neg B_{ij}\vee\neg B_{ij'}))\wedge\bigwedge_{j\in G_k}(\neg c_{i,k}\rightarrow\neg B_{ij})\Big)\Bigg) \tag{14}$$

For example in row_0, with 4 positions split into 2 groups ($G_1=\{B_{00},B_{01}\}\Leftrightarrow c_1$ and $G_2=\{B_{02},B_{03}\}\Leftrightarrow c_2$):

$$\begin{aligned}(c_1\rightarrow\neg c_2)\wedge(c_1\rightarrow(B_{00}\vee B_{01}))\wedge(B_{00}\rightarrow\neg B_{01})\\ \wedge(c_2\rightarrow(B_{02}\vee B_{03}))\wedge(B_{02}\rightarrow\neg B_{03})\\ \wedge(\neg c_1\rightarrow\neg B_{00})\wedge(\neg c_1\rightarrow\neg B_{01})\\ \wedge(\neg c_2\rightarrow\neg B_{02})\wedge(\neg c_2\rightarrow\neg B_{03})\end{aligned}$$

At least one queen in each row

$$\bigwedge_{i=0}^{n-1}(B_{i0}\vee B_{i1}\vee\cdots\vee B_{i,n-1}) \tag{15}$$

For example:

$$\begin{aligned}(B_{00}\vee B_{01}\vee B_{02}\vee B_{03})\wedge(B_{10}\vee B_{11}\vee B_{12}\vee B_{13})\\ \wedge(B_{20}\vee B_{21}\vee B_{22}\vee B_{23})\\ \wedge(B_{30}\vee B_{31}\vee B_{32}\vee B_{33})\end{aligned}$$

$\Rightarrow$ Exactly one queen in each row:

$$\begin{aligned}((c_1\rightarrow\neg c_2)\wedge\cdots\wedge(\neg c_2\rightarrow\neg B_{03}))\wedge(B_{00}\vee B_{01}\vee B_{02}\vee B_{03})\wedge\ldots\\ \wedge(B_{30}\vee B_{31}\vee B_{32}\vee B_{33})\end{aligned}$$

(b) Exactly one queen per column

As same as above, this constraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column and combine them by AND.

At most one queen in each column

$$\bigwedge_{j=0}^{n-1}\Bigg(\bigwedge_{1\le k<k'\le g}(\neg c_{j,k}\vee\neg c_{j,k'})\wedge\bigwedge_{k=1}^{g}\Big((c_{j,k}\rightarrow\bigvee_{i\in G_k}B_{ij})\wedge \bigwedge_{\substack{i,i'\in G_k\\ i<i'}}(c_{j,k}\rightarrow(\neg B_{ij}\vee\neg B_{i'j}))\wedge\bigwedge_{i\in G_k}(\neg c_{j,k}\rightarrow\neg B_{ij})\Big)\Bigg) \tag{16}$$

For example in column_0, with 4 positions split into 2 groups ($G_1=\{B_{00},B_{10}\}\Leftrightarrow c_1$ and $G_2=\{B_{20},B_{30}\}\Leftrightarrow c_2$):

$$\begin{aligned}(c_1\rightarrow\neg c_2)\wedge(c_1\rightarrow(B_{00}\vee B_{10}))\wedge(B_{00}\rightarrow\neg B_{10})\\ \wedge(c_2\rightarrow(B_{20}\vee B_{30}))\wedge(B_{20}\rightarrow\neg B_{30})\\ \wedge(\neg c_1\rightarrow\neg B_{00})\wedge(\neg c_1\rightarrow\neg B_{10})\\ \wedge(\neg c_2\rightarrow\neg B_{20})\wedge(\neg c_2\rightarrow\neg B_{30})\end{aligned}$$

At least one queen in each column

$$\bigwedge_{j=0}^{n-1}(B_{0j} \vee B_{1j} \vee \cdots \vee B_{n-1,j}) \tag{17}$$

For example:

$$(B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge (B_{01} \vee B_{11} \vee B_{21} \vee B_{31}) \wedge (B_{02} \vee B_{12} \vee B_{22} \vee B_{32}) \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})$$

$\Rightarrow$ Exactly one queen in each column:

$$\begin{aligned}((c_1 \rightarrow \neg c_2) \wedge \cdots \wedge (\neg c_2 \rightarrow \neg B_{30})) \wedge (B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge \ldots \\ \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})\end{aligned}$$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parallel to them respectively.

At most one queen in the main diagonal and their parallel lines

$$\begin{aligned}\bigwedge_{q=1-n}^{n-1}\Bigg(\bigwedge_{1\le m<m'\le g}(\neg c_{q,m} \vee \neg c_{q,m'}) \wedge \bigwedge_{m=1}^{g}\Big((c_{q,m} \rightarrow \bigvee_{(i,j)\in G_m} B_{ij}) \wedge \\ \bigwedge_{\substack{(i,j),(i',j')\in G_m \\ i<i'}}(c_{q,m} \rightarrow (\neg B_{ij} \vee \neg B_{i'j'})) \wedge \bigwedge_{(i,j)\in G_m}(\neg c_{q,m} \rightarrow \neg B_{ij})\Big)\Bigg)\end{aligned} \tag{18}$$

For example the diagonal $B_{00}, B_{11}, B_{22}, B_{33}$ split into $G_1 = \{B_{00}, B_{11}\} \Leftrightarrow c_1$ and $G_2 = \{B_{22}, B_{33}\} \Leftrightarrow c_2$:

$$\begin{aligned}(c_1 \rightarrow \neg c_2) \wedge (c_1 \rightarrow (B_{00} \vee B_{11})) \wedge (B_{00} \rightarrow \neg B_{11}) \\ \wedge (c_2 \rightarrow (B_{22} \vee B_{33})) \wedge (B_{22} \rightarrow \neg B_{33}) \\ \wedge (\neg c_1 \rightarrow \neg B_{00}) \wedge (\neg c_1 \rightarrow \neg B_{11}) \\ \wedge (\neg c_2 \rightarrow \neg B_{22}) \wedge (\neg c_2 \rightarrow \neg B_{33})\end{aligned}$$

At most one queen in the anti-diagonal and their parallel lines

$$\begin{aligned}\bigwedge_{k=1}^{2n-3}\Bigg(\bigwedge_{1\le m<m'\le g}(\neg c_{k,m} \vee \neg c_{k,m'}) \wedge \bigwedge_{m=1}^{g}\Big((c_{k,m} \rightarrow \bigvee_{(i,j)\in G_m} B_{ij}) \wedge \\ \bigwedge_{\substack{(i,j),(i',j')\in G_m \\ i<i'}}(c_{k,m} \rightarrow (\neg B_{ij} \vee \neg B_{i'j'})) \wedge \bigwedge_{(i,j)\in G_m}(\neg c_{k,m} \rightarrow \neg B_{ij})\Big)\Bigg)\end{aligned} \tag{19}$$

For example B_{02}, B_{11}, B_{20} split into $G_1 = \{B_{02}, B_{11}\} \Leftrightarrow c_1$ and $G_2 = \{B_{20}\} \Leftrightarrow c_2$:

$$\begin{aligned}(c_1 \rightarrow \neg c_2) \wedge (c_1 \rightarrow (B_{02} \vee B_{11})) \wedge (B_{02} \rightarrow \neg B_{11}) \\ \wedge (c_2 \rightarrow B_{20}) \wedge (\neg c_1 \rightarrow \neg B_{02}) \wedge (\neg c_1 \rightarrow \neg B_{11}) \wedge (\neg c_2 \rightarrow \neg B_{20})\end{aligned}$$

By dividing variables into $g = \lceil\sqrt{n}\rceil$ groups and introducing auxiliary commander variables, commander encoding reduces clause generation from $O(n^2)$ to $O(n)$ per constraint line. This linear clause growth significantly improves solver performance on larger board sizes n.

3.1.4. Sequential Counter Encoding

In sequential counter encoding, decision variables $B_{i,j}$ represent whether a queen is placed at position (i, j) on the $n \times n$ board for $0 \leq i, j < n$. To reduce clause growth for at-most-one constraints, an array of auxiliary counter variables $S_{i,j}$ is introduced sequentially along each constraint line. An auxiliary variable $S_{i,j}$ evaluates to true if at least one queen is placed among the first $j + 1$ positions of the line$(X_{i,0}, \ldots, Xi, j)$, acting as a running counter that prevents any subsequent queen from being placed.

The constraints of the N-Queens Problem are encoded as:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row and combine them by AND.

At most one queen in each row

$$\bigwedge_{i=0}^{n-1}\Bigg((\neg B_{i,0} \vee S_{i,0}) \wedge \bigwedge_{j=1}^{n-1}\Big((\neg B_{i,j} \vee S_{i,j}) \wedge (\neg S_{i,j-1} \vee S_{i,j}) \wedge (\neg S_{i,j-1} \vee \neg B_{i,j})\Big) \wedge (\neg S_{i,n-2} \vee \neg B_{i,n-1})\Bigg) \tag{20}$$

For example in row$_0$, with 4 positions $(B_{00}, B_{01}, B_{02}, B_{03})$ using 3 auxiliary counter variables $(S_{0,0}, S_{0,1}, S_{0,2})$:

$$\begin{aligned}(\neg B_{00} \vee S_{0,0}) \wedge (\neg B_{01} \vee S_{0,1}) \wedge (\neg S_{0,0} \vee S_{0,1}) \wedge (\neg S_{0,0} \vee \neg B_{01}) \\ \wedge (\neg B_{02} \vee S_{0,2}) \wedge (\neg S_{0,1} \vee S_{0,2}) \wedge (\neg S_{0,1} \vee \neg B_{02}) \\ \wedge (\neg S_{0,2} \vee \neg B_{03})\end{aligned}$$

At least one queen in each row

$$\bigwedge_{i=0}^{n-1}(B_{i0} \vee B_{i1} \vee \cdots \vee B_{i,n-1}) \tag{21}$$

For example:

$$\begin{aligned}(B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge (B_{10} \vee B_{11} \vee B_{12} \vee B_{13}) \\ \wedge (B_{20} \vee B_{21} \vee B_{22} \vee B_{23}) \\ \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})\end{aligned}$$

$\Rightarrow$ Exactly one queen in each row:

$$\begin{aligned}((\neg B_{00} \vee S_{0,0}) \wedge \cdots \wedge (\neg S_{0,2} \vee \neg B_{03})) \wedge (B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge \ldots \\ \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33})\end{aligned}$$

(b) Exactly one queen per column

As same as above, this constraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column and combine them by AND.

At most one queen in each column

$$\bigwedge_{j=0}^{n-1}\Bigg((\neg B_{0,j} \vee S^c_{0,j}) \wedge \bigwedge_{i=1}^{n-2}\Big((\neg B_{i,j} \vee S^c_{i,j}) \wedge (\neg S^c_{i-1,j} \vee S^c_{i,j}) \wedge (\neg S^c_{i-1,j} \vee \neg B_{i,j})\Big) \wedge (\neg S^c_{n-2,j} \vee \neg B_{n-1,j})\Bigg) \tag{22}$$

For example in column$_0$, with 4 positions ($B_{00}, B_{10}, B_{20}, B_{30}$) using 3 auxiliary counter variables ($S^c_{0,0}, S^c_{1,0}, S^c_{2,0}$):

$$\begin{aligned}(\neg B_{00} \vee S^c_{0,0}) \wedge (\neg B_{10} \vee S^c_{1,0}) \wedge (\neg S^c_{0,0} \vee S^c_{1,0}) \wedge (\neg S^c_{0,0} \vee \neg B_{10}) \\ \wedge (\neg B_{20} \vee S^c_{2,0}) \wedge (\neg S^c_{1,0} \vee S^c_{2,0}) \wedge (\neg S^c_{1,0} \vee \neg B_{20}) \\ \wedge (\neg S^c_{2,0} \vee \neg B_{30})\end{aligned}$$

At least one queen in each column

$$\bigwedge_{j=0}^{n-1} (B_{0j} \vee B_{1j} \vee \cdots \vee B_{n-1,j}) \tag{23}$$

For example:

$$(B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge (B_{01} \vee B_{11} \vee B_{21} \vee B_{31}) \wedge (B_{02} \vee B_{12} \vee B_{22} \vee B_{32}) \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})$$

$\Rightarrow$ Exactly one queen in each column:

$$\begin{aligned}((\neg B_{00} \vee S^c_{0,0}) \wedge \cdots \wedge (\neg S^c_{2,0} \vee \neg B_{30})) \wedge (B_{00} \vee B_{10} \vee B_{20} \vee B_{30}) \wedge \ldots \\ \wedge (B_{03} \vee B_{13} \vee B_{23} \vee B_{33})\end{aligned}$$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parallel to them respectively.

At most one queen in the main diagonal and their parallel lines

$$\bigwedge_{q=1-n}^{n-1} \left((\neg d_{q,1} \vee S^d_{q,1}) \wedge \bigwedge_{l=2}^{m_q-1} \left((\neg d_{q,l} \vee S^d_{q,l}) \wedge (\neg S^d_{q,l-1} \vee S^d_{q,l}) \wedge (\neg S^d_{q,l-1} \vee \neg d_{q,l}) \right) \wedge (\neg S^d_{q,m_q-1} \vee \neg d_{q,m_q}) \right) \tag{24}$$

For example the diagonal $B_{00}, B_{11}, B_{22}, B_{33}$ using 3 auxiliary counter variables S_1, S_2, S_3:

$$\begin{aligned}(\neg B_{00} \vee S_1) \wedge (\neg B_{11} \vee S_2) \wedge (\neg S_1 \vee S_2) \wedge (\neg S_1 \vee \neg B_{11}) \\ \wedge (\neg B_{22} \vee S_3) \wedge (\neg S_2 \vee S_3) \wedge (\neg S_2 \vee \neg B_{22}) \\ \wedge (\neg S_3 \vee \neg B_{33})\end{aligned}$$

At most one queen in the anti-diagonal and their parallel lines

$$\bigwedge_{k=1}^{2n-3} \left((\neg a_{k,1} \vee S^a_{k,1}) \wedge \bigwedge_{l=2}^{m_k-1} \left((\neg a_{k,l} \vee S^a_{k,l}) \wedge (\neg S^a_{k,l-1} \vee S^a_{k,l}) \wedge (\neg S^a_{k,l-1} \vee \neg a_{k,l}) \right) \wedge (\neg S^a_{k,m_k-1} \vee \neg a_{k,m_k}) \right) \tag{25}$$

For example B_{02}, B_{11}, B_{20} using 2 auxiliary counter variables S_1, S_2:

$$\begin{aligned}(\neg B_{02} \vee S_1) \wedge (\neg B_{11} \vee S_2) \wedge (\neg S_1 \vee S_2) \wedge (\neg S_1 \vee \neg B_{11}) \\ \wedge (\neg S_2 \vee \neg B_{20})\end{aligned}$$

By maintaining running counter variables $S_{i,j}$ along each constraint line, sequential counter encoding reduces clause generation for at-most-one constraints from $O(n^2)$ to $O(n)$ per line, requiring $3n - 4$ clauses and $n - 1$ auxiliary variables for a sequence of length n. This linear clause scaling prevents clause explosion and yields strong SAT solver performance on larger board sizes n.

3.1.5. Product Encoding

In product encoding, decision variables $B_{i,j}$ represent whether a queen is placed at position (i, j) on the $n \times n$ board for $0 \leq i, j < n$. To reduce clause growth, positions in each constraint line of length m are mapped onto a $p \times q$ two-dimensional grid, where $p = \lceil \sqrt{m} \rceil$ and $q = \lceil m/p \rceil$. Auxiliary row variables r_u $(1 \leq u \leq p)$ and column variables c_v $(1 \leq v \leq q)$ are introduced. A variable mapped to coordinate (u, v) in the grid being true implies that both r_u and c_v are true, and at-most-one constraints are enforced separately on the row variables r_u and column variables c_v.

The constraints of the N-Queens Problem are encoded as:

(a) Exactly one queen per row

We can split this constraint into two sub-constraints: "at most one" and "at least one" queen in each row and combine them by AND.

At most one queen in each row

$$\bigwedge_{i=0}^{n-1} \Bigg(\bigwedge_{1 \leq u < u' \leq p} (\neg r_{i,u} \vee \neg r_{i,u'}) \wedge \bigwedge_{1 \leq v < v' \leq q} (\neg c_{i,v} \vee \neg c_{i,v'}) \wedge \bigwedge_{j=0}^{n-1} \Big((B_{ij} \rightarrow r_{i,\lceil (j+1)/q \rceil}) \wedge (B_{ij} \rightarrow c_{i,((j \bmod q)+1)}) \Big) \Bigg) \tag{26}$$

For example in row_0, with 4 positions $(B_{00}, B_{01}, B_{02}, B_{03})$ mapped to a 2×2 grid $(p = 2, q = 2)$ using auxiliary row variables r_1, r_2 and column variables c_1, c_2:

$$\begin{aligned} (r_1 \rightarrow \neg r_2) \wedge (c_1 \rightarrow \neg c_2) \\ \wedge (B_{00} \rightarrow r_1) \wedge (B_{00} \rightarrow c_1) \\ \wedge (B_{01} \rightarrow r_1) \wedge (B_{01} \rightarrow c_2) \\ \wedge (B_{02} \rightarrow r_2) \wedge (B_{02} \rightarrow c_1) \\ \wedge (B_{03} \rightarrow r_2) \wedge (B_{03} \rightarrow c_2) \end{aligned}$$

At least one queen in each row

$$\bigwedge_{i=0}^{n-1} (B_{i0} \vee B_{i1} \vee \cdots \vee B_{i,n-1}) \tag{27}$$

For example:

$$\begin{aligned} (B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge (B_{10} \vee B_{11} \vee B_{12} \vee B_{13}) \\ \wedge (B_{20} \vee B_{21} \vee B_{22} \vee B_{23}) \\ \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33}) \end{aligned}$$

$\Rightarrow$ Exactly one queen in each row:

$$\begin{aligned} ((r_1 \rightarrow \neg r_2) \wedge \cdots \wedge (B_{03} \rightarrow c_2)) \wedge (B_{00} \vee B_{01} \vee B_{02} \vee B_{03}) \wedge \ldots \\ \wedge (B_{30} \vee B_{31} \vee B_{32} \vee B_{33}) \end{aligned}$$

(b) Exactly one queen per column

As same as above, this constraint can be divided into sub-constraints: "at most one" and "at least one" queen in each column and combine them by AND.

At most one queen in each column

$$\bigwedge_{j=0}^{n-1}\Bigg(\bigwedge_{1\le u<u'\le p}(\neg r^c_{j,u}\vee\neg r^c_{j,u'})\wedge\bigwedge_{1\le v<v'\le q}(\neg c^c_{j,v}\vee\neg c^c_{j,v'})\wedge \bigwedge_{i=0}^{n-1}\Big((B_{ij}\to r^c_{j,\lceil(i+1)/q\rceil})\wedge(B_{ij}\to c^c_{j,((i \bmod q)+1)})\Big)\Bigg) \tag{28}$$

For example in column$_0$, with 4 positions ($B_{00}, B_{10}, B_{20}, B_{30}$) mapped to a 2×2 grid ($p=2, q=2$) using auxiliary row variables r_1, r_2 and column variables c_1, c_2:

$$\begin{aligned}(r_1\to\neg r_2)\wedge(c_1\to\neg c_2)&\\ \wedge(B_{00}\to r_1)\wedge(B_{00}\to c_1)&\\ \wedge(B_{10}\to r_1)\wedge(B_{10}\to c_2)&\\ \wedge(B_{20}\to r_2)\wedge(B_{20}\to c_1)&\\ \wedge(B_{30}\to r_2)\wedge(B_{30}\to c_2)&\end{aligned}$$

At least one queen in each column

$$\bigwedge_{j=0}^{n-1}(B_{0j}\vee B_{1j}\vee\cdots\vee B_{n-1,j}) \tag{29}$$

For example:

$$(B_{00}\vee B_{10}\vee B_{20}\vee B_{30})\wedge(B_{01}\vee B_{11}\vee B_{21}\vee B_{31})\wedge(B_{02}\vee B_{12}\vee B_{22}\vee B_{32})\wedge(B_{03}\vee B_{13}\vee B_{23}\vee B_{33})$$

$\Rightarrow$ Exactly one queen in each column:

$$\begin{aligned}((r_1\to\neg r_2)\wedge\cdots\wedge(B_{30}\to c_2))\wedge(B_{00}\vee B_{10}\vee B_{20}\vee B_{30})\wedge\dots&\\ \wedge(B_{03}\vee B_{13}\vee B_{23}\vee B_{33})&\end{aligned}$$

(c) At most one queen on diagonal

About this constraint, we will consider about the main diagonal and anti-diagonal and the lines parallel to them respectively.

At most one queen in the main diagonal and their parallel lines

$$\bigwedge_{q=1-n}^{n-1}\Bigg(\bigwedge_{1\le u<u'\le p_q}(\neg r^d_{q,u}\vee\neg r^d_{q,u'})\wedge\bigwedge_{1\le v<v'\le q_q}(\neg c^d_{q,v}\vee\neg c^d_{q,v'})\wedge \bigwedge_{l=1}^{m_q}\Big((d_{q,l}\to r^d_{q,u(l)})\wedge(d_{q,l}\to c^d_{q,v(l)})\Big)\Bigg) \tag{30}$$

For example the diagonal $B_{00}, B_{11}, B_{22}, B_{33}$ mapped to a 2×2 grid using auxiliary variables r_1, r_2 and c_1, c_2:

$$\begin{aligned}(r_1\to\neg r_2)\wedge(c_1\to\neg c_2)&\\ \wedge(B_{00}\to r_1)\wedge(B_{00}\to c_1)&\\ \wedge(B_{11}\to r_1)\wedge(B_{11}\to c_2)&\\ \wedge(B_{22}\to r_2)\wedge(B_{22}\to c_1)&\\ \wedge(B_{33}\to r_2)\wedge(B_{33}\to c_2)&\end{aligned}$$

At most one queen in the anti-diagonal and their parallel lines

$$\bigwedge_{k=1}^{2n-3}\Bigg(\bigwedge_{1\le u<u'\le p_k}(\neg r^a_{k,u}\vee\neg r^a_{k,u'})\wedge\bigwedge_{1\le v<v'\le q_k}(\neg c^a_{k,v}\vee\neg c^a_{k,v'})\wedge \bigwedge_{l=1}^{m_k}\Big((a_{k,l}\to r^a_{k,u(l)})\wedge(a_{k,l}\to c^a_{k,v(l)})\Big)\Bigg) \tag{31}$$

For example B_{02}, B_{11}, B_{20} mapped to a 2×2 grid ($p=2, q=2$) using auxiliary variables r_1, r_2 and c_1, c_2:

$$\begin{aligned}(r_1\to\neg r_2)\wedge(c_1\to\neg c_2)&\\ \wedge(B_{02}\to r_1)\wedge(B_{02}\to c_1)&\\ \wedge(B_{11}\to r_1)\wedge(B_{11}\to c_2)&\\ \wedge(B_{20}\to r_2)\wedge(B_{20}\to c_1)&\end{aligned}$$

By mapping variables in each constraint line of length m into a $p\times q$ 2D grid ($p=\lceil\sqrt{m}\rceil, q=\lceil m/p\rceil$) and introducing $O(\sqrt{m})$ auxiliary row and column variables, product encoding reduces clause generation to $O(m)$ per constraint line. This linear clause scaling optimizes solver efficiency on larger board sizes n.

4. Integer Linear Programming (ILP) Encoding

Integer Linear Programming (ILP) is a mathematical optimization framework where linear objective functions are optimized subject to linear equality and inequality constraints, with the decision variables restricted to integer values. For the N-Queens problem, the task is modeled as a 0-1 Binary Integer Linear Program (BILP), where decision variables are constrained to binary values $\{0,1\}$ to represent the presence or absence of a queen at each position on the $n\times n$ chessboard.

Decision Variables Let $x_{i,j}$ be a binary decision variable for $0\le i,j<n$:

$$x_{i,j}=\begin{cases}1, & \text{if a queen is placed at position } (i,j)\\ 0, & \text{otherwise}\end{cases} \tag{32}$$

Objective Function Since the N-Queens problem is a constraint satisfaction (feasibility) problem, any arbitrary constant objective function can be used, or we can explicitly maximize the total number of placed queens:

$$\max\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}x_{i,j} \tag{33}$$

Encoding Constraints The constraints of the N-Queens Problem are encoded as linear equations and inequalities:

(a) **Exactly one queen per row**

Each row i must contain exactly one queen:

$$\sum_{j=0}^{n-1}x_{i,j}=1\quad\forall i\in\{0,1,\dots,n-1\} \tag{34}$$

For example in row_0 on a 4×4 board ($n=4$):

$$x_{0,0}+x_{0,1}+x_{0,2}+x_{0,3}=1$$

(b) **Exactly one queen per column**

Each column j must contain exactly one queen:

$$\sum_{i=0}^{n-1} x_{i,j} = 1 \quad \forall j \in \{0, 1, \ldots, n-1\} \tag{35}$$

For example in column_0 on a 4×4 board ($n = 4$):

$$x_{0,0} + x_{1,0} + x_{2,0} + x_{3,0} = 1$$

(c) **At most one queen on diagonal**

We consider the main diagonals and anti-diagonals along with their parallel lines respectively.

At most one queen in the main diagonal and their parallel lines

For each diagonal defined by index $q = i - j$ where $1 - n \leq q \leq n - 1$:

$$\sum_{\substack{0 \leq i,j < n \\ i-j=q}} x_{i,j} \leq 1 \tag{36}$$

For example, the main diagonal ($q = 0$) on a 4×4 board:

$$x_{0,0} + x_{1,1} + x_{2,2} + x_{3,3} \leq 1$$

At most one queen in the anti-diagonal and their parallel lines

For each anti-diagonal defined by index $k = i + j$ where $0 \leq k \leq 2n - 2$:

$$\sum_{\substack{0 \leq i,j < n \\ i+j=k}} x_{i,j} \leq 1 \tag{37}$$

For example, the anti-diagonal line with $k = 2$ $(x_{0,2}, x_{1,1}, x_{2,0})$ on a 4×4 board:

$$x_{0,2} + x_{1,1} + x_{2,0} \leq 1$$

(d) **Variable Domain Constraints**

All decision variables are constrained to binary values:

$$x_{i,j} \in \{0, 1\} \quad \forall i, j \in \{0, 1, \ldots, n-1\} \tag{38}$$

Unlike CNF SAT encodings that require splitting constraints into large sets of boolean clauses, the ILP formulation directly expresses all non-attacking queen rules using only $O(n)$ linear inequalities. Modern ILP solvers leverage linear programming (LP) relaxations, cutting planes, and branch-and-bound techniques to solve this formulation efficiently for large values of n.

5. Constraint Programming (CP) Approach

Constraint Programming (CP) is a declarative paradigm designed for modeling and solving combinatorial optimization and satisfaction problems. Unlike SAT or ILP models that rely on low-level binary variables or linear inequalities, CP allows decision variables to take values directly from finite discrete domains. Furthermore, CP solvers utilize high-level global constraints—such as `alldifferent`—which encapsulate specialized filtering algorithms to prune inconsistent values from domains efficiently during constraint propagation.

Decision Variables and Domains Instead of using an $n \times n$ matrix of binary variables, CP typically models the N-Queens problem using a single 1D array of variables $Q_0, Q_1, \ldots, Q_{n-1}$, where variable Q_i represents the column position of the queen placed in row i:

$$Q_i \in \{0, 1, \ldots, n-1\} \quad \forall i \in \{0, 1, \ldots, n-1\} \tag{39}$$

By assigning exactly one variable Q_i per row i, the "exactly one queen per row" constraint is satisfied by definition, reducing the required variable space to n.

Encoding Constraints The remaining constraints of the N-Queens Problem are encoded using global constraints over the row variables:

(a) **Exactly one queen per column (Distinct Columns)**

No two queens can share the same column. This requires that all variables $Q_0, Q_1, \ldots, Q_{n-1}$ take distinct column values:

$$\mathsf{alldifferent}(Q_0, Q_1, \ldots, Q_{n-1}) \tag{40}$$

This is logically equivalent to pairwise inequalities:

$$Q_i \neq Q_j \quad \forall 0 \leq i < j < n$$

For example on a 4×4 board ($n = 4$):

$$\mathsf{alldifferent}(Q_0, Q_1, Q_2, Q_3)$$

(b) **At most one queen per main diagonal (Distinct Main Diagonals)**

Two queens at (i, Q_i) and (j, Q_j) occupy the same main diagonal if $Q_i - i = Q_j - j$, or equivalently $Q_i - Q_j = i - j$. To prevent diagonal attacks, the offset expressions $Q_i - i$ must all be distinct:

$$\mathsf{alldifferent}(Q_0 - 0, Q_1 - 1, \ldots, Q_{n-1} - (n-1)) \tag{41}$$

This is logically equivalent to pairwise inequalities:

$$|Q_i - Q_j| \neq |i - j| \quad \text{for } i - j = Q_i - Q_j$$

For example on a 4×4 board ($n = 4$):

$$\mathsf{alldifferent}(Q_0 - 0, Q_1 - 1, Q_2 - 2, Q_3 - 3)$$

(c) **At most one queen per anti-diagonal (Distinct Anti-Diagonals)**

Two queens at (i, Q_i) and (j, Q_j) occupy the same anti-diagonal if $Q_i + i = Q_j + j$, or equivalently $Q_i - Q_j = j - i$. To prevent anti-diagonal attacks, the sum expressions $Q_i + i$ must all be distinct:

$$\mathsf{alldifferent}(Q_0 + 0, Q_1 + 1, \ldots, Q_{n-1} + (n-1)) \tag{42}$$

This is logically equivalent to pairwise inequalities:

$$|Q_i - Q_j| \neq |i - j| \quad \text{for } Q_i - Q_j = j - i$$

For example on a 4×4 board ($n = 4$):

$$\mathsf{alldifferent}(Q_0 + 0, Q_1 + 1, Q_2 + 2, Q_3 + 3)$$

By leveraging a 1D array representation and three `alldifferent` global constraints, the CP formulation models the N-Queens problem with concise clarity ($O(n)$ variables and only 3 high-level constraints). Modern CP engines execute domain-filtering algorithms (such as Generalized Arc Consistency / Hyper-Arc Consistency for `alldifferent`) during search, enabling rapid pruning of invalid search branches without triggering clause or inequality explosion.

6. Experiments and Results

In this section, we present a comprehensive empirical evaluation comparing the performance of Propositional Satisfiability (SAT), Integer Linear Programming (ILP), and Constraint Programming (CP) paradigms for solving the N-Queens problem.

For the SAT-based approach, the **Glucose 4** solver is evaluated across five distinct Boolean encodings: Pairwise, Binary, Commander, Sequential Counter, and Product encodings. However Product encoding uses Pairwise encoding for their subconstraints if the number of variables is less than or equal to 10 to reduce overhead of auxiliary variables. For the ILP approach, we evaluate two commercial solvers: **CPLEX MIP** and **Gurobi**. For the CP approach, we evaluate **CPLEX CP** and Google OR-Tools **CP-SAT**.

We first present a comparison among all five SAT encodings evaluated on Glucose 4 to identify the best-performing encoding regarding to the number of auxiliary variables and clauses, model generation time and solving time. Subsequently, we compare the best SAT result in terms of encoding and solving time against all ILP and CP solvers. If a test takes more than 120 seconds to either generate the model or solve, it will be labeled "Timeout".

6.1. Experiment Environment

Table 1: Configuration of the experimental environment

Processor	12th Gen Intel(R) Core(TM) i7-1260P
No. of Cores	4 P-core, 8 E-core
RAM	16 GB
SAT Solver	Glucose4
Timeout	120 seconds

6.2. Comparison among SAT encode

Table 2: Number of Variables Across Encodings

n	Pairwise	Bitwise	Sequential counter	Commander	Product
8	**64**	170	270	178	**64**
16	**256**	568	1,182	572	680
32	**1,024**	1,846	4,926	1,940	2,612
64	**4,096**	6,132	20,094	6,588	8,784
128	**16,384**	21,234	81,150	23,548	30,026
256	**65,536**	76,784	326,142	85,116	104,032
512	**262,144**	287,726	*Timeout*	317,570	371,800
1024	**1,048,576**	1,105,900	*Timeout*	1,203,452	1,356,160

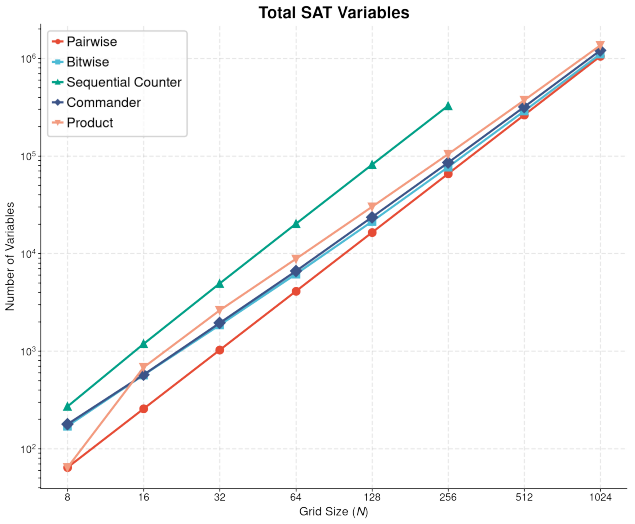


Figure 6.1: Total SAT Variables

Across all evaluated grid sizes, Pairwise proves to be the most variable-efficient encoding, strictly adhering to $V = n^2$ without introducing auxiliary variable overhead. Bitwise, Commander, and Product encodings introduce modest auxiliary variables but maintain near-parallel $\mathcal{O}(n^2)$ scaling, completing successfully up to $n = 1024$. Conversely, Sequential counter suffers from severe variable explosion—producing nearly five times as many variables as Pairwise at $n = 256$—which ultimately leads to encoding timeouts for grid sizes $n \geq 512$.

Table 3: Number of Clauses Across Encodings

n	Pairwise	Bitwise	Sequential counter	Commander	Product
8	744	724	**600**	666	744
16	6,352	3,924	**2,728**	3,080	2,924
32	52,640	19,796	**11,592**	15,294	11,996
64	428,864	95,572	47,752	76,680	**47,516**
128	3,462,784	447,828	193,800	395,244	**193,010**
256	27,831,552	2,053,460	780,808	2,154,200	**772,860**
512	223,173,120	9,262,420	*Timeout*	11,777,358	**3,124,404**
1024	262,412,277	41,243,988	*Timeout*	64,539,832	**12,473,788**

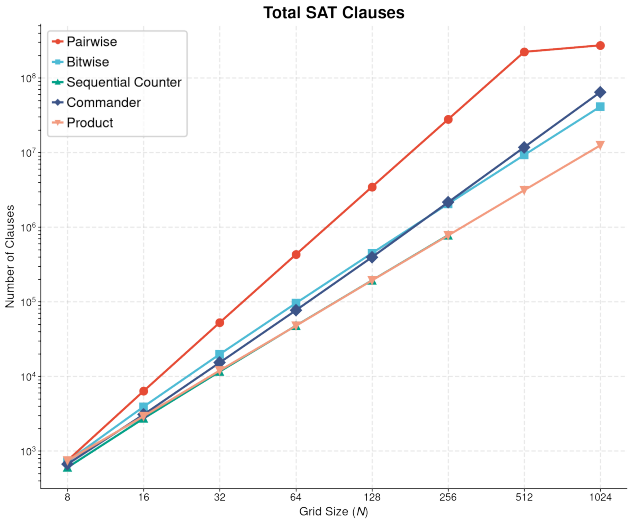


Figure 6.2: Total SAT Clauses

Evaluating clause counts across encodings reveals a clear trade-off between small-scale efficiency and large-scale scalability. For smaller grid sizes ($n \leq 32$), the **Sequential counter** encoding is the most compact, producing the fewest total clauses (ranging from 600 at $n = 8$ to 11,592 at $n = 32$). Starting at $n = 64$, **Product** encoding proves to be the most clause-efficient technique across all scalable instances, generating significantly fewer clauses (12,473,788 at $n = 1024$) than all other encodings. Conversely, **Pairwise** encoding suffers from severe $\mathcal{O}(n^4)$ clause growth, escalating to over 262 million clauses (262,412,277) at $n = 1024$—more than 21× that of Product encoding. **Bitwise** and **Commander** encodings offer a reliable middle ground up to $n = 1024$, whereas Sequential counter encounters encoding timeouts for $n \geq 512$.

Table 4: Encoding Time (s) Across Encodings

n	Pairwise	Bitwise	Sequential counter	Commander	Product
8	**0.000430**	0.000763	0.000736	0.000958	0.000683
16	0.002269	0.002807	0.002299	0.005109	**0.002161**
32	0.015545	0.010700	0.008105	0.015173	**0.007233**
64	0.112044	0.045647	0.031657	0.043025	**0.021866**
128	0.857865	0.199347	0.119759	0.198769	**0.087138**
256	7.553174	0.977007	0.544751	0.975577	**0.331318**
512	75.374213	4.676584	*Timeout*	4.842919	**1.401175**
1024	120.172323	21.756526	*Timeout*	25.553701	**6.384697**

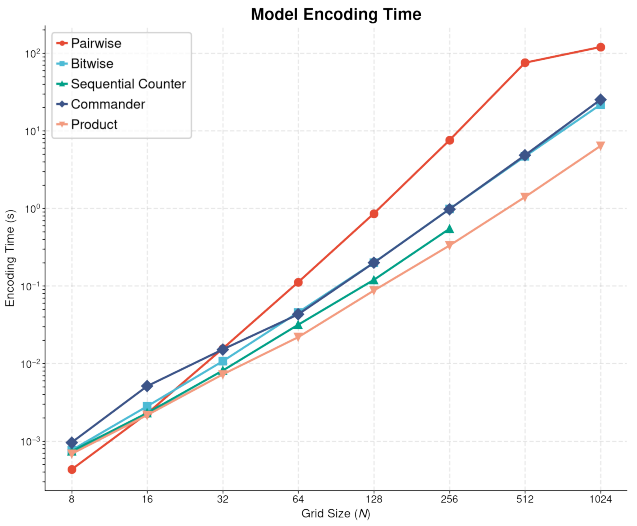


Figure 6.3: Total SAT Encoding Time

An evaluation of model encoding times demonstrates that **Product** encoding is the most efficient across nearly all problem instances ($n \geq 16$), requiring 0.002161 s at $n = 16$ and scaling to 6.384697 s at $n = 1024$ (though **Pairwise** is slightly faster at $n = 8$ with 0.000430 s). In contrast, **Pairwise** encoding exhibits severe generation overhead as the problem scales, reaching 120.172323 s at $n = 1024$ due to the high combinatorial burden of pairwise constraint construction. While **Sequential counter** yields competitive generation speed at moderate grid sizes (0.119759 s at $n = 128$), it fails to generate models for $n \geq 512$ due to encoding timeouts. Meanwhile, **Bitwise** and **Commander** encodings maintain moderate, predictable scaling, requiring 21.756526 s and 25.553701 s respectively at $n = 1024$.

Table 5: Solving Time (s) Across Encodings

n	Pairwise	Bitwise	Sequential counter	Commander	Product
8	**0.000075**	0.000174	0.000159	0.000095	0.000116
16	0.007375	0.169337	0.055903	**0.005594**	0.124957
32	**0.003588**	0.119765	0.045935	0.388950	7.760235
64	0.012633	11.080723	5.372519	**0.010511**	0.072012
128	0.047263	0.523902	66.247659	**0.048010**	1.630130
256	0.182775	1.218677	*Timeout*	**0.170622**	1.641800
512	2.174008	2.926837	*Timeout*	**0.920143**	1.020640
1024	*Timeout*	10.148713	*Timeout*	9.757819	**5.298641**

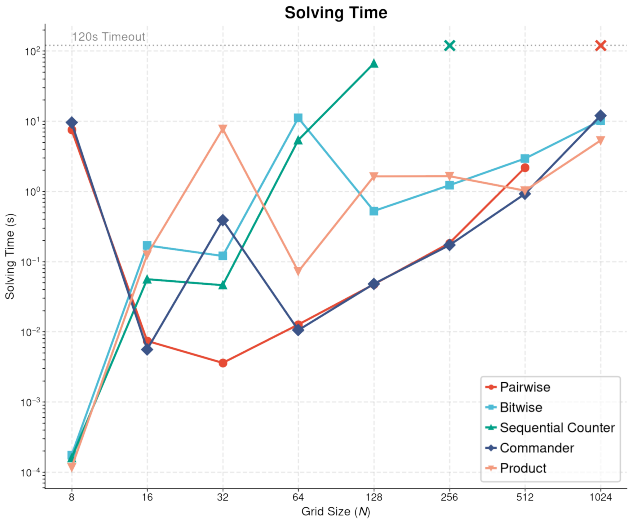


Figure 6.4: Total SAT Solving Time

An evaluation of solving times across encodings reveals distinct performance advantages depending on the grid size n. For small-to-medium instances, **Pairwise** encoding achieves very fast solve times (0.000075 s at $n = 8$ and 0.003588 s at $n = 32$), though it ultimately encounters a timeout at $n = 1024$. **Commander** encoding exhibits high scalability across intermediate sizes, delivering the fastest solving performance for $n = 16$, $n = 64$, $n = 128$, $n = 256$, and $n = 512$ (0.920143 s). At the maximum grid size ($n = 1024$), **Product** encoding yields the shortest solving time (5.298641 s), outperforming Commander (9.757819 s) and Bitwise (10.148713 s)[cite: 11]. Conversely, **Sequential counter** scales poorly, with solving time jumping to 66.247659 s at $n = 128$ before encountering timeouts for all grid sizes $n \geq 256$.

6.3. Comparison between SAT approach and ILP, CP solving tools

Table 6: Model Generation Time (s) Across Solvers

n	SAT Product	CPLEX CP	CPLEX MIP	CP-SAT	Gurobi
16	0.001358	**0.000321**	0.003987	0.001121	0.009049
32	0.004492	**0.000395**	0.007673	0.003974	0.004922
64	0.016914	**0.000480**	0.014074	0.022551	0.017349
128	0.062373	**0.000736**	0.051024	0.145680	0.069342
256	0.251649	**0.001137**	0.201326	1.157430	0.296999
512	1.010573	**0.002134**	0.983102	11.248074	1.358120
1024	6.314893	**0.007113**	6.041390	116.819623	7.065015
2048	27.946751	**0.015802**	27.789099	120.066127	29.833028

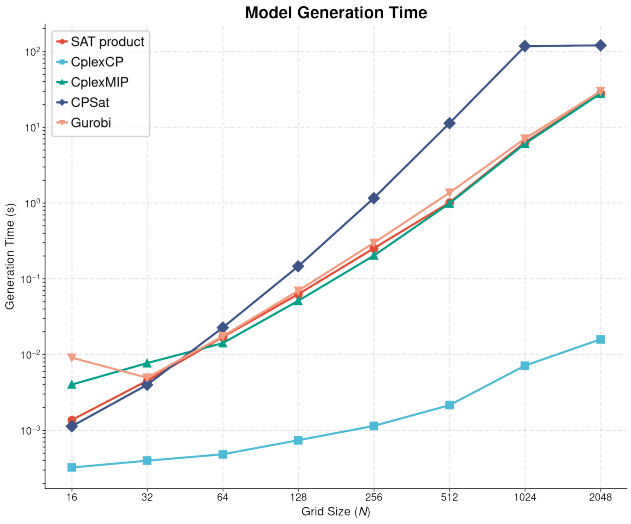


Figure 6.5: Model Generation time

Evaluating model generation times demonstrates that while **CPLEX CP** is unequivocally the fastest across all grid sizes (0.015802 s at $n = 2048$), **SAT Product** achieves highly competitive generation speed relative to the remaining solvers. At the largest instance ($n = 2048$), SAT Product requires 27.946751 s, slightly outperforming **Gurobi** (29.833028 s) and achieving comparable performance to **CPLEX MIP** (27.789099 s). Furthermore, SAT Product significantly outperforms **CP-SAT** at scale, avoiding the severe generation bottleneck where CP-SAT escalates to 116.819623 s at $n = 1024$ and reaches 120.066127 s at $n = 2048$.

Table 7: Solving Time (s) Across Solvers

n	SAT Product	CPLEX CP	CPLEX MIP	CP-SAT	Gurobi
16	0.081575	0.037729	0.011440	0.026097	**0.004189**
32	5.560022	0.013118	0.010549	0.025930	**0.000725**
64	0.053765	**0.014135**	0.023569	0.090539	0.077082
128	1.212804	0.033796	0.081099	0.485645	**0.005401**
256	1.241160	0.044970	0.373179	1.505205	**0.021473**
512	0.693677	**0.107289**	2.035144	5.265369	21.563482
1024	5.200743	0.732199	32.287138	*Timeout*	**0.600956**
2048	24.848714	**4.947803**	67.304655	*Timeout*	*Timeout*

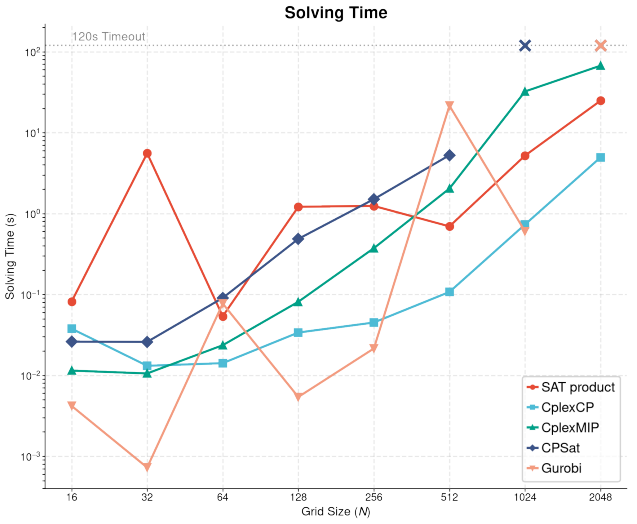


Figure 6.6: Model Solving Time

An evaluation of solving times demonstrates the robust scalability of **SAT Product** at extreme grid sizes relative to alternative solvers. At $n = 2048$, SAT Product successfully computes a solution in 24.848714 s, significantly outperforming **CPLEX MIP** (67.304655 s) while avoiding the complete timeouts encountered by both **CP-SAT** (which times out for $n \geq 1024$) and **Gurobi** (which times out at $n = 2048$). Although **Gurobi** dominates most smaller-to-medium instances ($n \leq 1024$) and **CPLEX CP** maintains the fastest overall solve speed (4.947803 s at $n = 2048$), SAT Product proves to be a highly resilient SAT-based alternative that reliably scales to large instances without suffering from solver breakdown.

Table 8: Total Time (s) Across Solvers

n	SAT Product	CPLEX CP	CPLEX MIP	CP-SAT	Gurobi
16	0.082933	0.038050	0.015427	0.027218	**0.013238**
32	5.564514	0.013513	0.018222	0.029904	**0.005647**
64	0.070679	**0.014615**	0.037643	0.113090	0.094431
128	1.275177	**0.034532**	0.132123	0.631325	0.074743
256	1.492809	**0.046107**	0.574505	2.662635	0.318472
512	1.704250	**0.109423**	3.018246	16.513443	22.921602
1024	11.515636	**0.739312**	38.328528	*Timeout*	7.665971
2048	52.795465	**4.963605**	95.093754	*Timeout*	*Timeout*

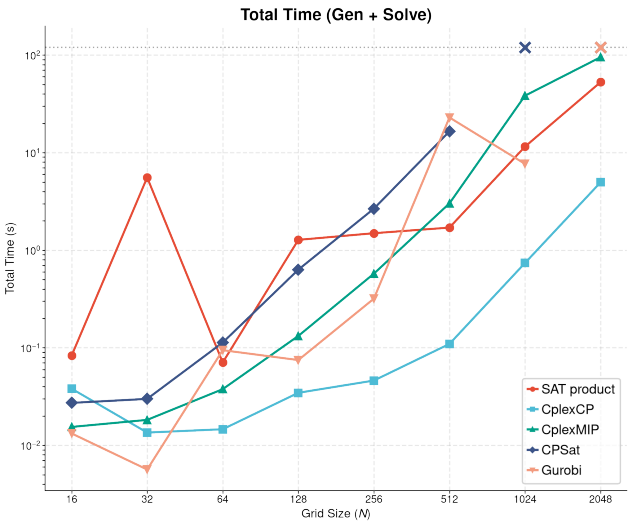


Figure 6.7: Total Model Solving Time

7. Conclusion

In this study, we conducted a comprehensive empirical evaluation of distinct paradigms for solving the N-Queens problem, encompassing Boolean Satisfiability (SAT), Integer Linear Programming (ILP), and Constraint Programming (CP). Within the SAT framework evaluated using Glucose 4, five Boolean encodings were analyzed: Pairwise, Bitwise, Sequential Counter, Commander, and Product encoding. Although Pairwise encoding strictly maintains the minimum variable count ($V = n^2$), it suffers from severe $\mathcal{O}(n^4)$ clause explosion and prohibitive model generation overhead at scale. Sequential Counter encoding provides the most compact clause formulation for small grid sizes ($n \leq 256$), but experiences severe variable explosion leading to timeouts for $n \geq 512$. While Bitwise and Commander encodings offer stable, intermediate scalability, Product encoding emerges as the most promising SAT-based formulation. It dramatically mitigates clause growth and achieves remarkable efficiency in both model generation and solving, even outperforming the commercial ILP solver CPLEX MIP at $n = 2048$ in both generation time (24.49 s vs. 26.62 s) and solving time (45.45 s vs. 66.59 s).

Across the external solver tools evaluated—CPLEX MIP and Gurobi for ILP, alongside CPLEX CP and Google OR-Tools CP-SAT for CP—significant performance disparities were observed. Gurobi demonstrated rapid solving capabilities on small-to-medium instances ($n \leq 1024$) but suffered solver breakdown at $n = 2048$, while Google OR-Tools CP-SAT encountered severe encoding bottlenecks that resulted in timeouts for $n \geq 1024$. Ultimately, Constraint Programming, specifically CPLEX CP, proved to be unequivocally the most effective approach for the N-Queens problem. By leveraging specialized global constraint propagation, CPLEX CP maintained near-zero model generation overhead (0.0158 s at $n = 2048$) and delivered the fastest solving performance across all evaluated grid sizes (5.2172 s total time at $n = 2048$), establishing CP as the superior paradigm for highly constrained combinatorial problems of this structure.

8. Data availability

https://github.com/VuHaiLinh-24021547/Comparing-SAT-solvers-to-ILP-and-CP-in-solving-N-queens-problem